20/06/2023

Convolutional Neural Networks

- Vertical edge detection

$$\begin{bmatrix} 3 & 0 & 1 & 2 & 7 & 4 \\ 1 & 5 & 8 & 9 & 3 & 1 \\ 2 & 7 & 2 & 5 & 1 & 3 \\ 0 & 1 & 3 & 1 & 7 & 8 \\ 4 & 2 & 1 & 6 & 2 & 8 \\ 2 & 4 & 5 & 2 & 3 & 9 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} -5 & -4 & 0 & 8 \\ -10 & -2 & 2 & 3 \\ 0 & -2 & -4 & -7 \\ -3 & -2 & -3 & -16 \end{bmatrix}$$

3×3 filter

4×4

- Padding

Size of image: $n\times n$, size of filter: $f\times f$, final size: $(n-f+1)\times(n-f+1)$ (without padding)

$(n+2p-f+1)\times(n+2p-f+1)$ (with padding)

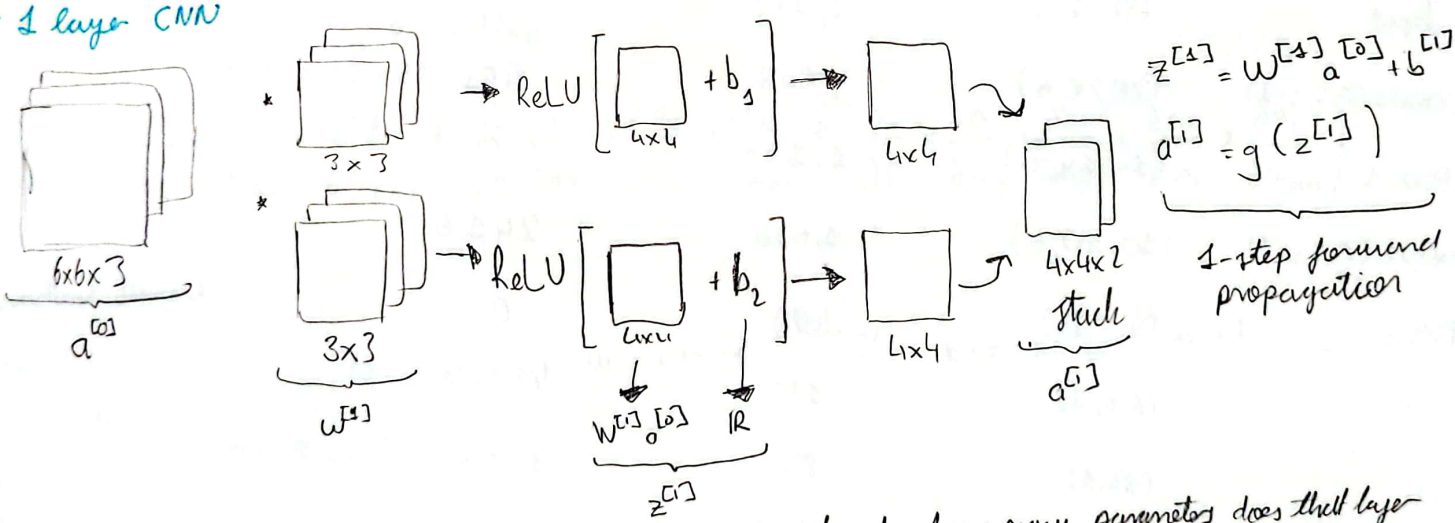
valid: no padding

same: pad so the size will be the same $p=\frac{f-1}{2}$

- Strided convolution

Take $\underbrace{\text{two}}_{s}$ steps horizontally instead of one step. Output size: $\left\lfloor \frac{n+2p-f}{s}+1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s}+1 \right\rfloor$

- 1 layer CNN



- If you have 10 $3\times3\times3$ filters in one layer of a neural network, how many parameters does that layer have? $3\times3\times3 = 27$ params + 1 bias $= 28$ $\quad 28\times10 = 280$

Note: Dimensions of the input doesn't affect the number of params. This makes CNNs less prone to overfitting

- Pooling layers

- Reduces the size of representation
- Speed up computation
- To make some of the feature detectors more robust

— max pooling

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

$\Rightarrow$

9	2
6	3

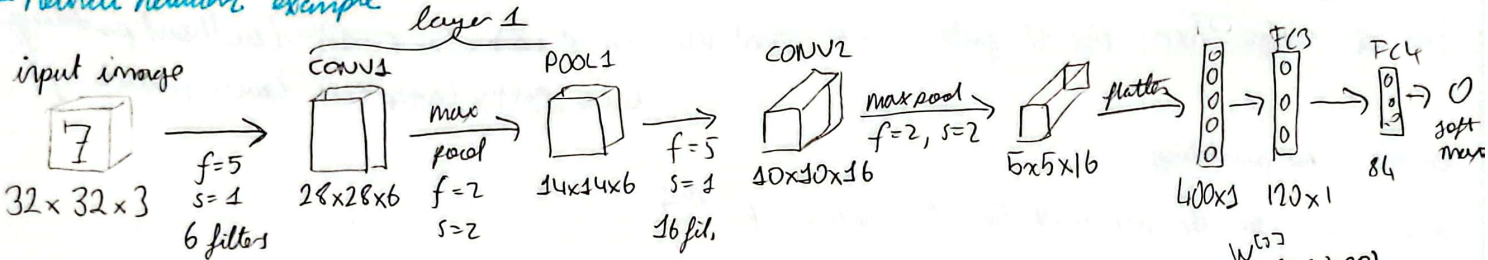
$f = 2,\ s = 2$

max of 2×2 region

if there are multiple channels apply max-pooling separately

no parameters to learn !

— neural network example



$W^{[3]}$ (120, 400)

$b^{[3]}$ (120)

Common pattern : CONV - POOL - CONV - POOL - FC - FC - FC - SOFTMAX

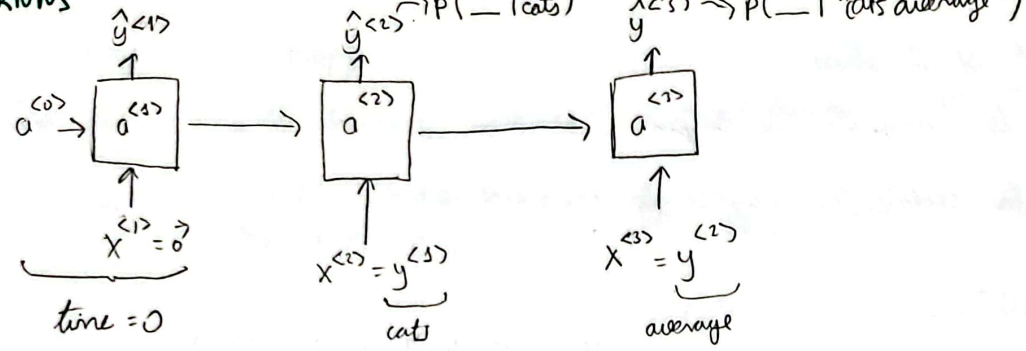
	Activation shape	Activation size	# of parameters
Input	(32, 32, 3)	3,072	0
CONV1 (f=5, s=1)	(28, 28, 6)	4,704	456 (5×5×3+1)×6
POOL1	(14, 14, 6)	1,176	0
CONV2 (f=5, s=1)	(10, 10, 16)	1,600	2416
POOL2	(5, 5, 16)	400	0
FC3	(120, 1)	120	48120
FC4	(84, 1)	84	10164 (120 × 84 + 84)
softmax	(10, 1)	10	850

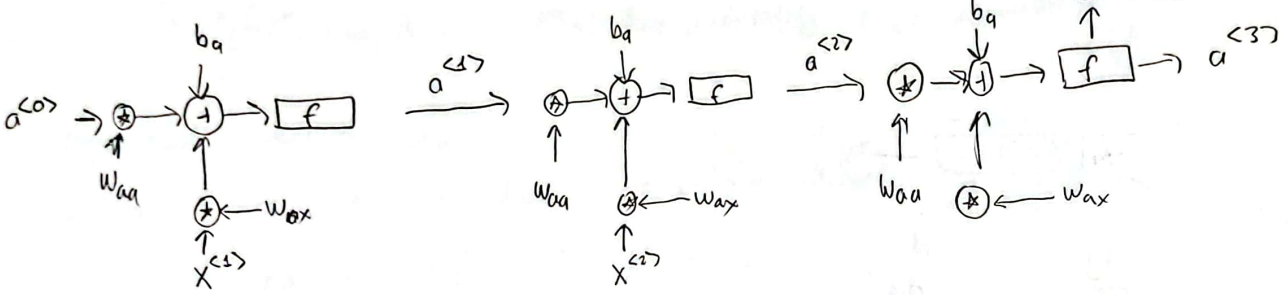
— Why convolutions

- Parameter sharing : a feature detector that's useful in one part of image is probably is useful in another part
- Sparsity of connections : in each layer, each output value depends only on a small number of inputs

1×1 convolution : the shrink # of channels

RNNs





$$a^{<t>} = \text{function}(W_{aa} * a^{<t-1>} + W_{ax} * x^{<t>} + b_a) \qquad 0 = \text{softmax}(W_{ya} * a^{<3>} + b_y)$$

$L \to 0 \to a_3 \to W_{aa}$

$\quad\quad \to a_2 \to W_{aa}$

$\quad\quad\quad \to a_1 \to W_{aa}$

$$\frac{\partial L}{\partial W_{aa}} = \sum_{i=1} \frac{\partial L}{\partial 0} \times \frac{\partial 0}{\partial a_i} \times \frac{\partial a_i}{\partial W_{aa}}$$

$$\frac{\partial L}{\partial W_{aa}} = \left(\frac{\partial L}{\partial 0} \times \frac{\partial 0}{\partial a_3} \times \frac{\partial a_3}{\partial W_{aa}}\right) + \left(\frac{\partial L}{\partial 0} \times \frac{\partial 0}{\partial a_3} \times \frac{\partial a_3}{\partial a_2} \times \frac{\partial a_2}{\partial W_{aa}}\right) + \left(\frac{\partial L}{\partial 0} \times \frac{\partial 0}{\partial a_3} \times \frac{\partial a_3}{\partial a_2} \times \frac{\partial a_2}{\partial a_1} \times \frac{\partial a_1}{\partial W_{aa}}\right)$$

Gradient Descent

Repeat {

$$w := w - \alpha \frac{dJ(w)}{dw}$$

}

For $J(w,b) \Rightarrow$ $\quad w := w - \alpha \frac{\partial J(w,b)}{\partial w}$ $\quad b := b - \alpha \frac{\partial J(w,b)}{\partial b}$

Computation Graph

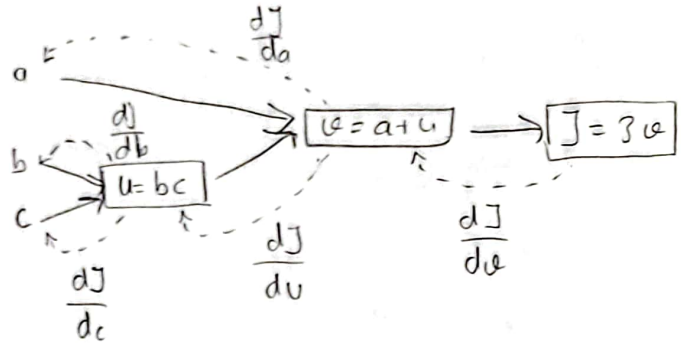
Operations of a NN consists of 2 steps:

1. Forward propagation to compute the output
2. Backward propagation to update the weights to minimize error

- $J(a,b,c) = 3(a+b*c)$

$u = bc \qquad v = a+u \qquad J = 3v$

if $a=5, b=3, c=2$ $\quad u=6$, $v=11$, $J=33$



(1) $\frac{dJ}{da} = \frac{d(3v)}{dv} = 3$ $\quad J = 3v$, $v = 11 \xrightarrow{\text{little inc.}} 11.001$ $\quad J: 33 \longrightarrow 33.003$ $\quad J$ changes by 3×0.001, $\frac{dJ}{dv}$

(2) $\frac{dJ}{da}$ $\quad a=5 \to 5.001$, $v=11 \to 11.001$, $J=33 \to 33.003$ } if we increase a by 0.001, J inc. by 0.003

$\frac{dJ}{da} = \frac{0.003}{0.001} = 3$

chain rule:

$\frac{dJ}{da} = \frac{dJ}{dv} \times \frac{dv}{da} = 3$ ($\frac{dJ}{dv} = 3$, $\frac{dv}{da} = 1$)

(3) $u=6 \to 6.001$, $v=11 \to 11.001$, $J=33 \to 33.003$ } if we inc. u by 0.001 $J \uparrow 0.003$

$\frac{dJ}{du} = \frac{dJ}{dv} \times \frac{dv}{du} = 3$ ($\frac{dJ}{dv} = 3$, $\frac{dv}{du} = 1$)

(4) $b=3 \to 3.001$, $u=6 \to 6.002$, $v=11 \to 11.002$, $J=33 \to 33.006$ } increase b by 0.001, J increased by 0.006

$\frac{dJ}{db} = \frac{dJ}{dv} \times \frac{dv}{du} \times \frac{du}{db} = \frac{dJ}{du} \times \frac{du}{db} = 6$ ($\frac{dJ}{dv} = 3$, $\frac{dv}{du} = 1$, $\frac{du}{db} = 2$, $\frac{dJ}{du} = 3$)

(5) $c=2 \to 2.001$, $u=6 \to 6.003$, $v=11 \to 11.003$, $J=33 \to 33.009$ } $c \uparrow 0.001 \Rightarrow J \uparrow 0.009$

$\frac{dJ}{dc} = \frac{dJ}{dv} \times \frac{dJ}{du} \times \frac{du}{dc} = 9$ ($\frac{dJ}{dv} = 3$, $\frac{dJ}{du} = 1$, $\frac{du}{dc} = 3$)

Loss Functions

— Binary cross entropy

Cost for a single training instance:

$$c(\theta) = \begin{cases} -\log(\hat{p}) & \text{if } y=1 \\ -\log(1-\hat{p}) & \text{if } y=0 \end{cases}$$

Cost function over the whole training set:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(\hat{p}^{(i)}\right) + \left(1-y^{(i)}\right)\log\left(1-\hat{p}^{(i)}\right)\right]$$

$$-\left(p^*(x)\log p(x) + (1-p^*(x))\log(1-p(x))\right)$$

— Cross entropy

$$H(P^*|P) = -\sum_i P^*(i)\log P(i)$$

$P^*(i)$: true probability $P(i)$: predicted probability i : class num

— Categorical cross entropy

$-\log P(x)$ x is the class that is 1 in our target

— Softmax

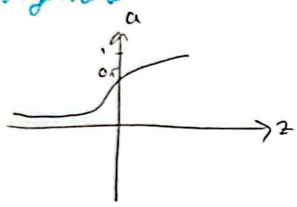
$$\mathcal{L}^{<t>}\left(\underbrace{\hat{y}^{<t>}}_{y^{pred}}, y^{<t>}\right) = -\sum_i y_i^{<t>}\log\hat{y}_i^{<t>}$$

— Triplet

$$\max\left(\|f^a - f^p\|_2^2 - \|f^a - f^n\|_2^2 + m, 0\right)$$

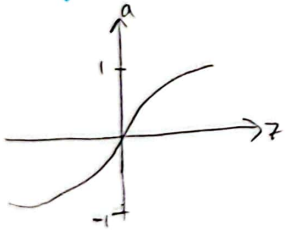
Activation Functions

— Sigmoid



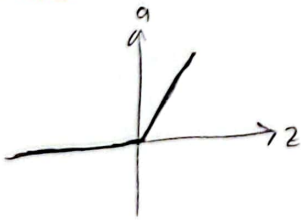
$$a = \frac{1}{1+e^{-z}} = \frac{e^z}{e^z+1} \in (0,1) \qquad g'(z) = a(1-a)$$

— Tangent hyperbolic

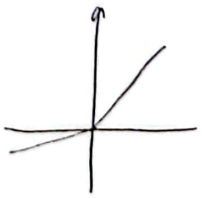


$$a = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \qquad g'(z) = 1-a^2$$

— ReLU



$$a = \begin{cases} \max(0,z), & z \geq 0 \\ 0 & z < 0 \end{cases}$$



Leaky ReLU